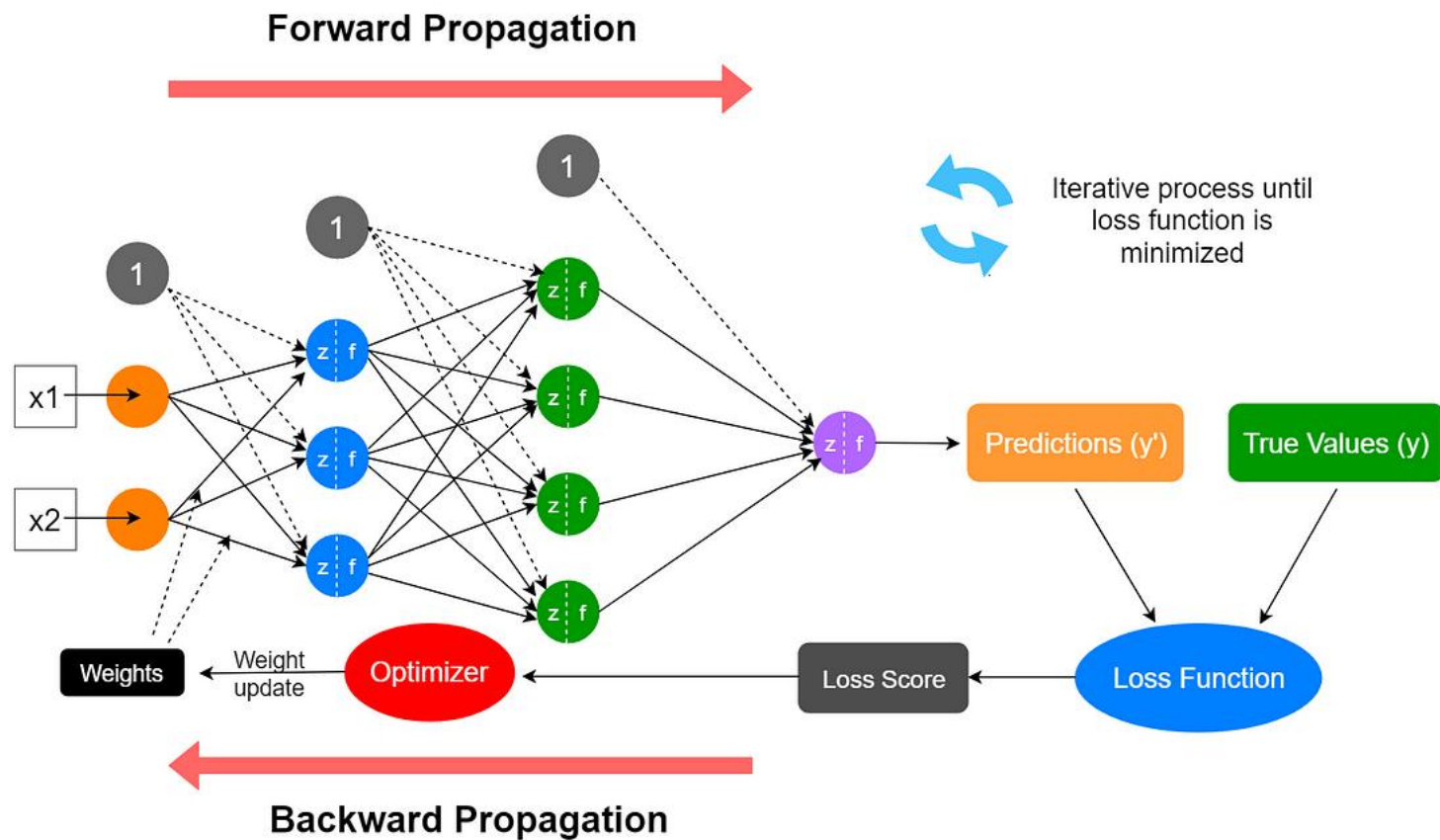


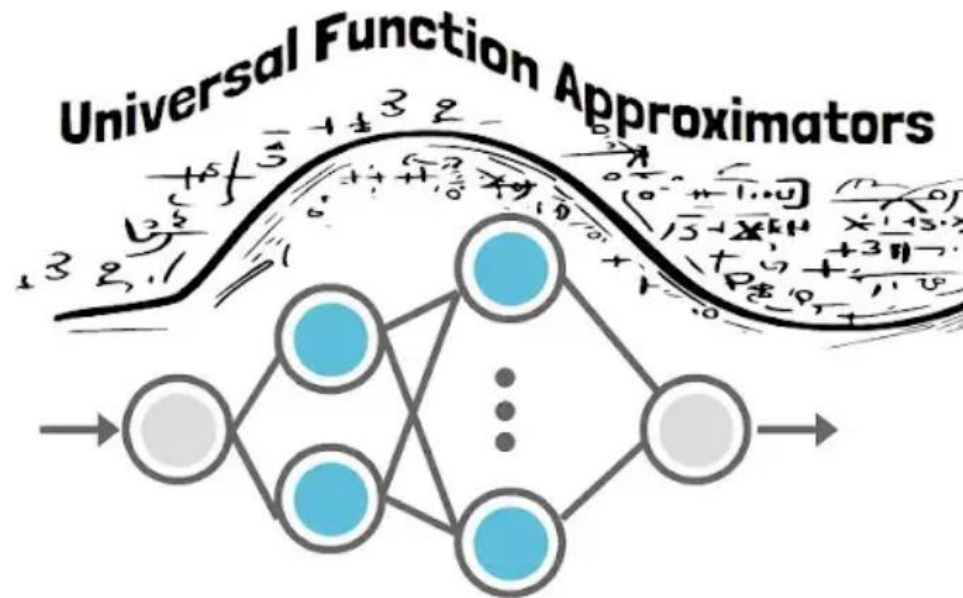
Artificial Neural Networks



Outline

- Introduction to Artificial Neural Networks
- Neuron: Fundamental Building Block of a NN
- Building a NN with Neurons
- Training Loop
- Regularization Techniques
- NN Architectures

Acknowledgement : some slides are adapted from Lecture of [MIT intro deep learning](#) and [LLM Chronicles](#)



Introduction to Artificial Neural Networks

Unique Powers of Neural Networks



- **Universal Function Approximation**

- **Approximate any continuous function**, making them versatile for diverse applications
- Neural networks are **composed differentiable functions**
 - *Self-driving cars map sensor inputs to steering decisions*
 - *Voice assistants convert speech waveforms into text commands*



- **Learning Complex Relationships**

- **Identify and learn intricate, nonlinear patterns** in data
 - *Banks detect fraudulent transactions across millions of data points*
 - *Recommendation engines predict user preferences from browsing behavior*



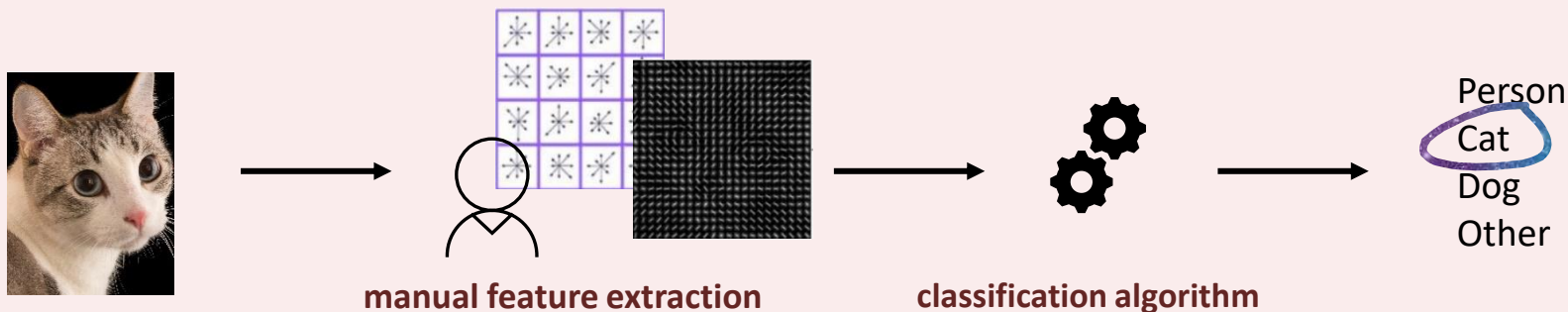
- **Feature Extraction**

- **Automatically extract and learn important features** from raw data
 - *Medical imaging detects tumors in X-rays without explicit programming*
 - *NLP models extract meaning and context from unstructured text*

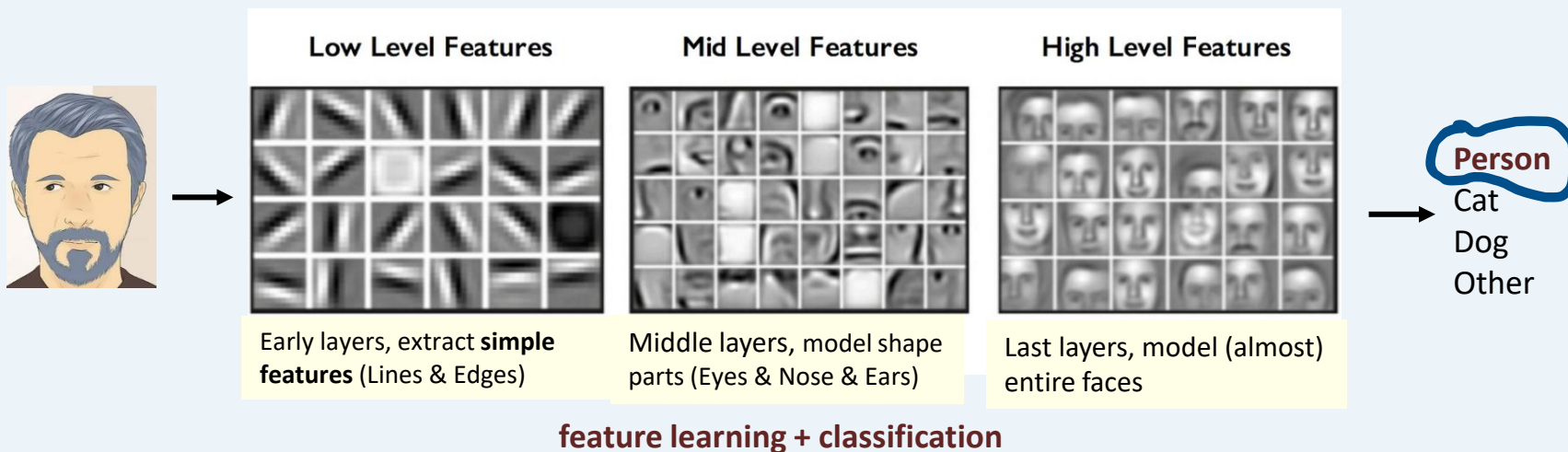


Learning Features Directly from Data

Traditional ML requires **hand-engineered features** — time consuming, brittle, and not scalable in practice



Deep Learning automatically **learns features directly from raw data** through many layers of abstraction — no manual engineering needed

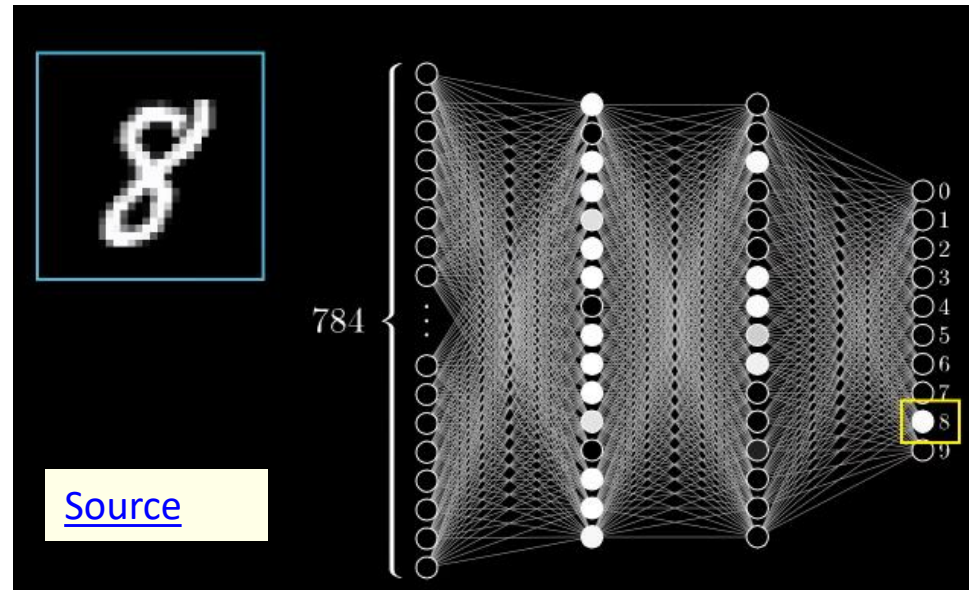


NN for recognizing handwritten digits

Network Structure

- **Input** — 784 neurons, one per pixel in a 28×28 image
- **Hidden** — 2 layers × 16 neurons; detect edges → shapes → digits
- **Output** — 10 neurons (digits 0–9); brightest neuron = prediction

[Source](#)



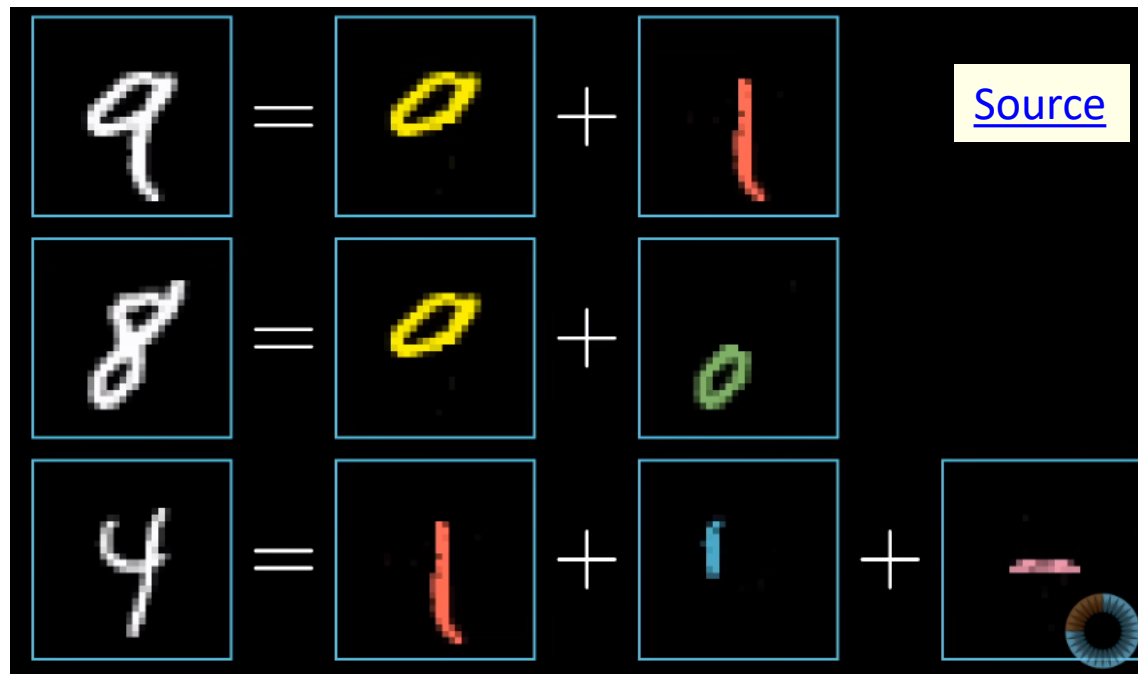
How Information Flows

- **Weight** - determines the importance of each connection between neurons (high weight = stronger influence)
- **Bias** - control how easily the neuron is activated (high bias = fires more readily)
- **Activation function** e.g., Sigmoid squishes (weights × inputs + bias) to 0–1, setting activation strength

Key Takeaway

Training = adjusting thousands of weights and biases until the network reliably maps pixel patterns to the correct digit

Example patterns - Recognizing handwritten digits

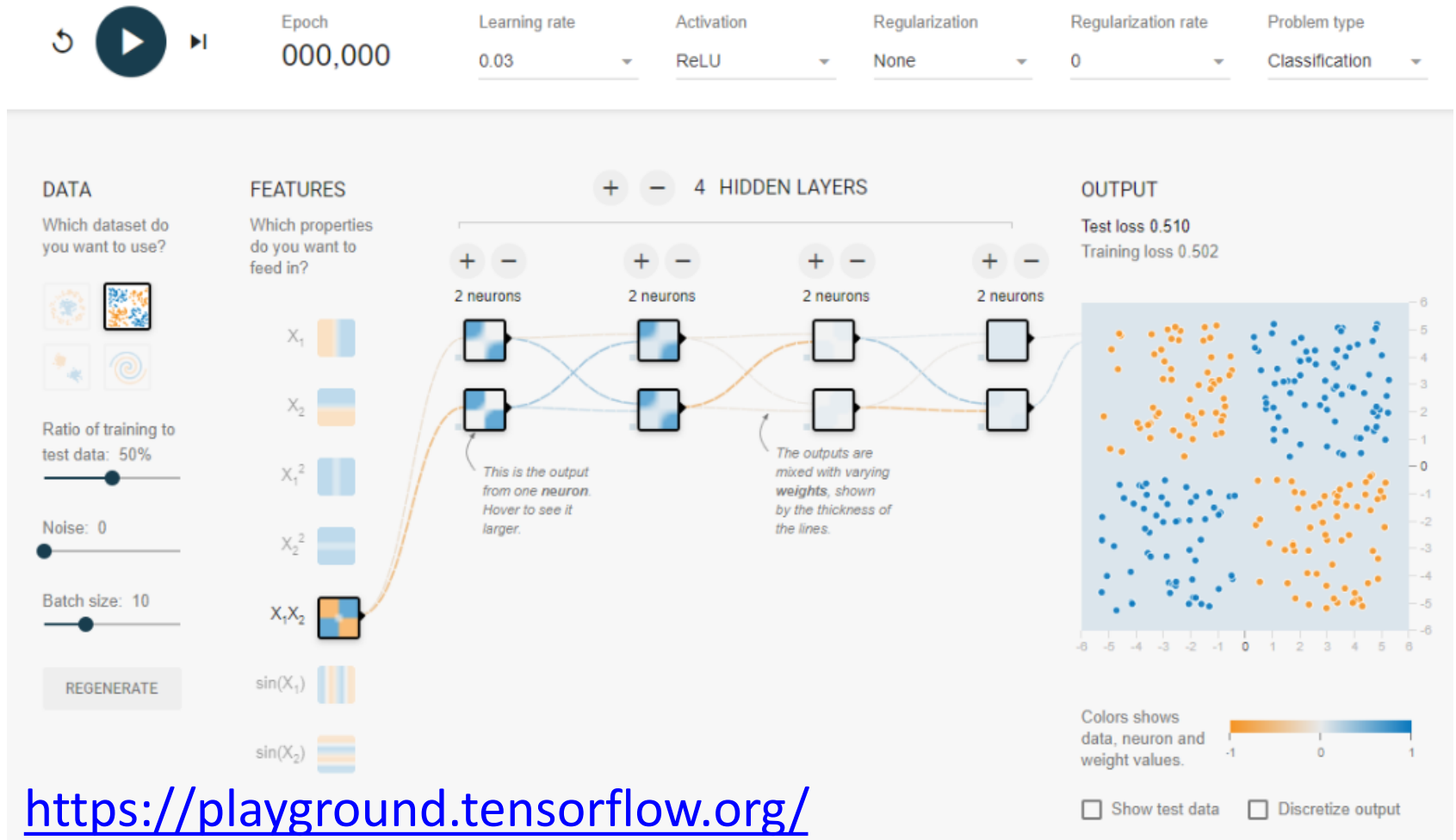


What Each Layer Learns

- **Layer 1** Edges — horizontal bars, vertical strokes, diagonals, small curves
- **Layer 2** Shapes — closed loops, open hooks, crossbars, intersections
- **Output** Digits — e.g. “8” = two stacked loops; “7” = top bar + diagonal

These patterns emerge from training — nobody tells the network to look for loops or edges.

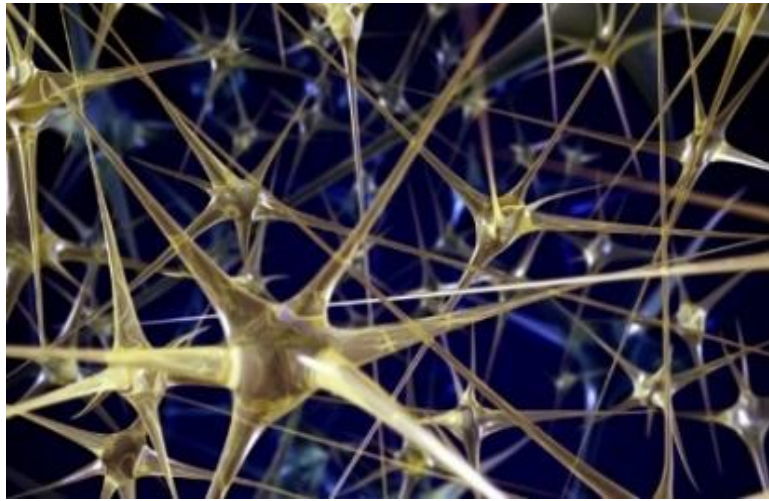
Fundamental Concepts - Visualizing a Neural Network



<https://playground.tensorflow.org/>

Key takeaway

A neural network learns by passing data forward, measuring error, and adjusting weights backward — then proving it generalizes to data it has never seen.



Neuron: Fundamental Building Block of a NN

Neuron

- A neuron = basic computational unit of a neural network

What a neuron does

- Computes a **weighted sum** of its inputs
- Adds a **bias** term
- Applies an **activation function** to produce the output

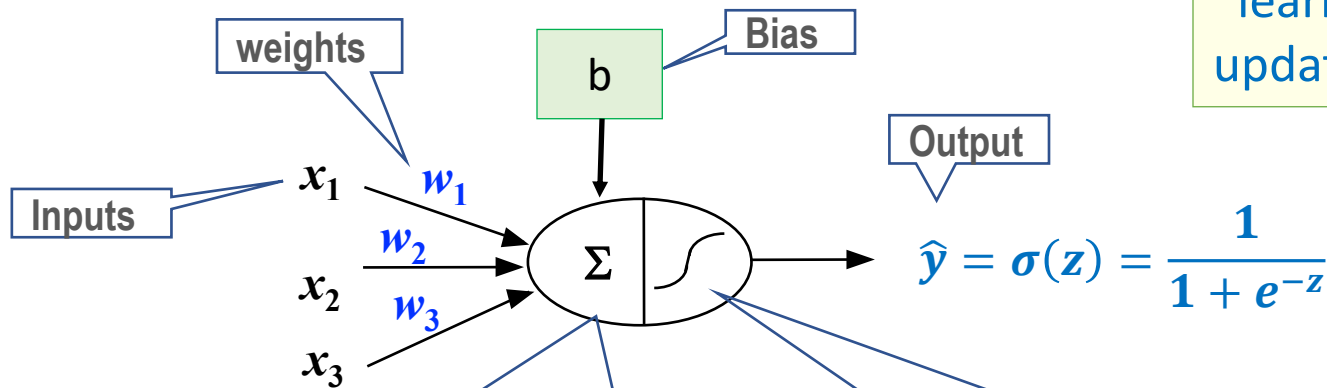
How neurons are organized

- Grouped into **layers** that process data in parallel
- **Input layer** → receives raw data
- **Hidden layers** → learn patterns through transformations
- **Output layer** → produces predictions or classifications

Neuron: the structural building block of NN

A neuron receives inputs, assigns each input an importance using a **weight**, adds a **bias**, then applies an **activation function** to produce an output

Weights and bias are learnable parameters updated during training



Sum function (linear combination of inputs):

$$z = x^T w + b = x_1 * w_1 + x_2 * w_2 + x_3 * w_3 + b$$

Non-linear Activation function (e.g. sigmoid)

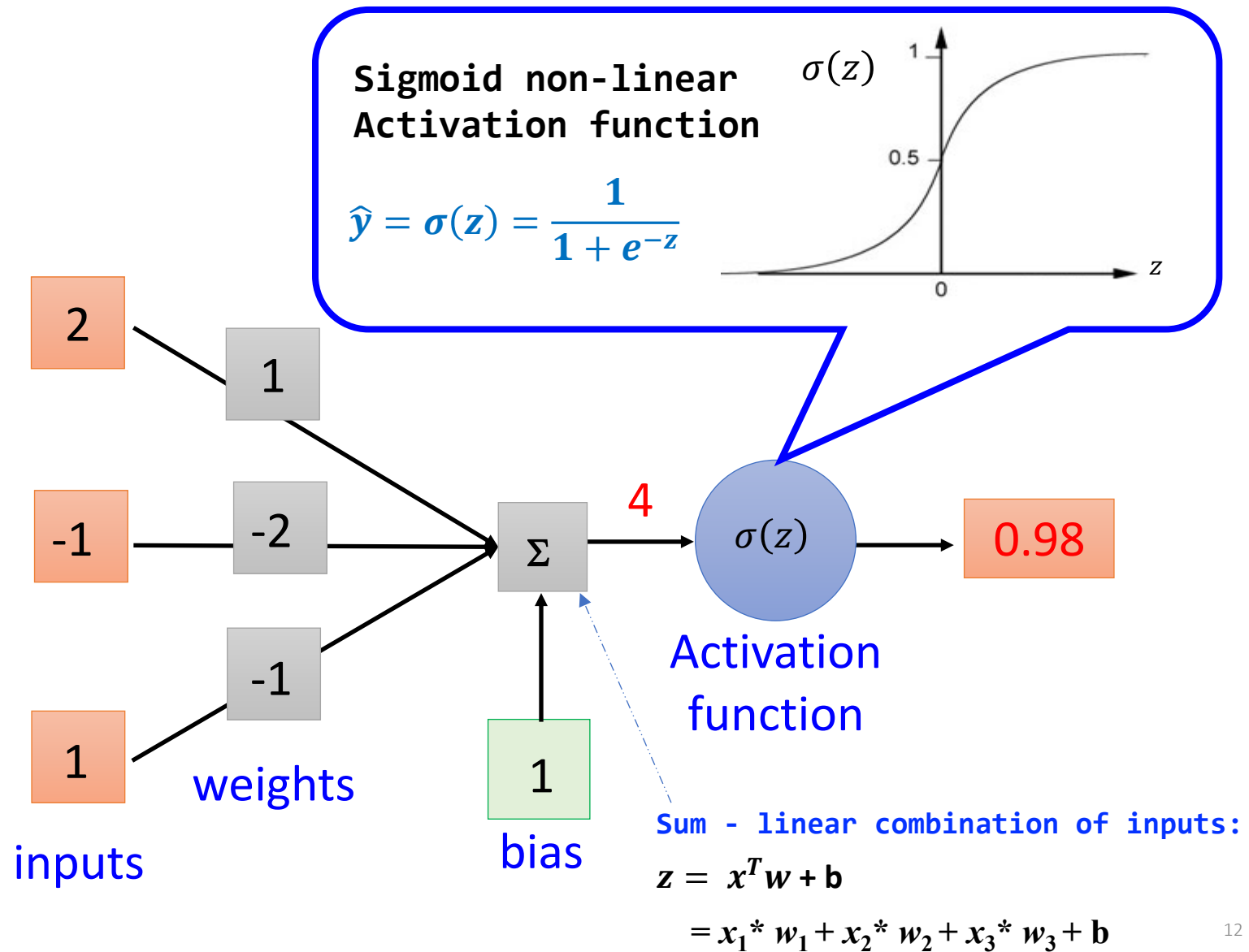
Where: $x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$: input features

$w = \begin{bmatrix} w_1 \\ w_2 \\ w_3 \end{bmatrix}$: weights control input importance

b : bias term that shifts the activation threshold

z : pre-activation value

Example - Perceptron model for Logistic regression

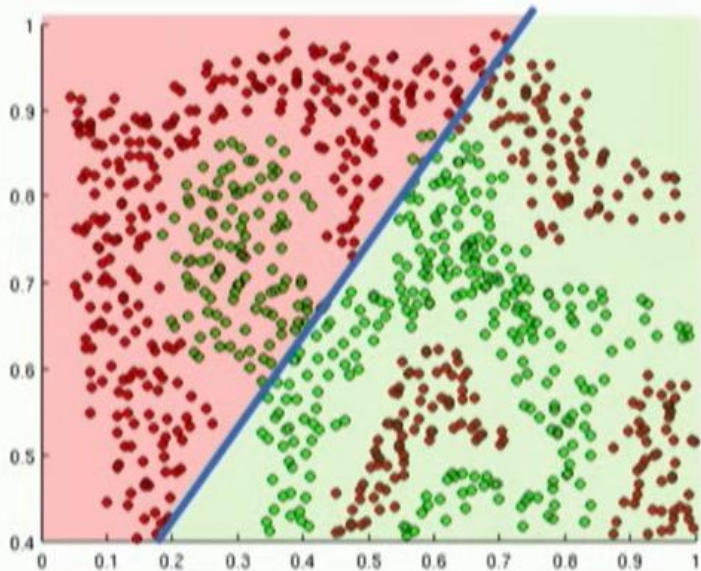


Activation function

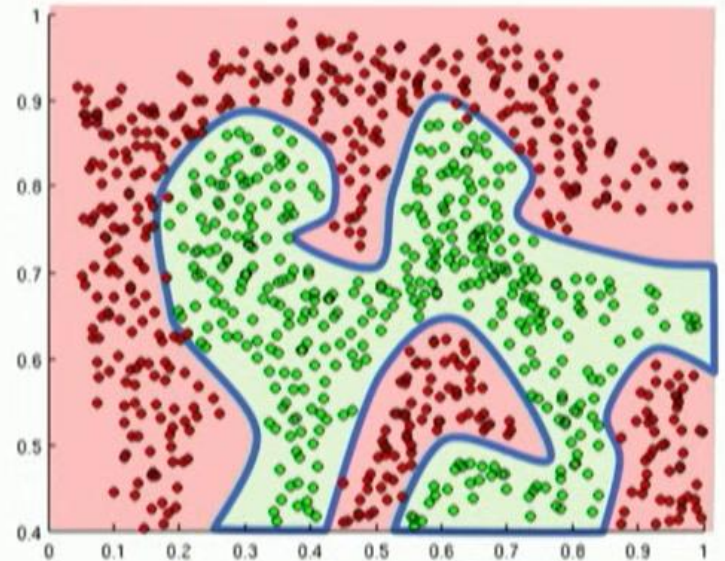
- **Add nonlinearity** — Allow neural networks to learn complex patterns beyond straight-line relationships
 - Enable deep learning: without nonlinear activations, multiple layers = linear model
- **Support gradient-based learning:** activation functions are differentiable, allowing backpropagation to compute gradients
- **Control output behavior:** Output-layer activations define the prediction format, such as probabilities or real values. (e.g., sigmoid → probability in $[0, 1]$)

Importance of Activation Functions

- Without activation functions, stacking layers is just repeated linear algebra — the network can only **draw straight decision boundaries** no matter how deep it is



Linear only → straight-line decisions, regardless of depth

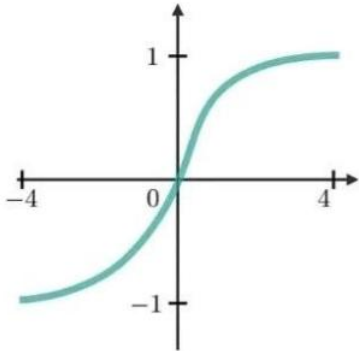


Nonlinear → can approximate any complex function

Activation functions of a neuron

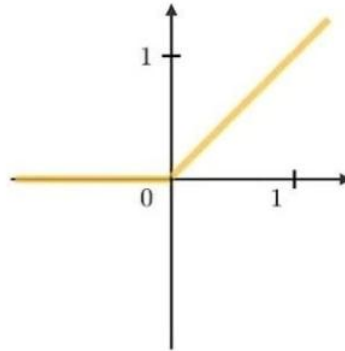
Hyperbolic Tangent (Tanh)

$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$



Rectified Linear Unit (ReLU)

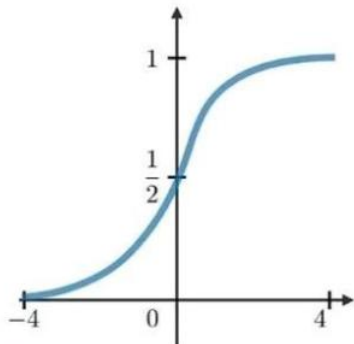
$$g(z) = \max(0, z)$$



The best activation function depends on the task, data, and layer position

Sigmoid

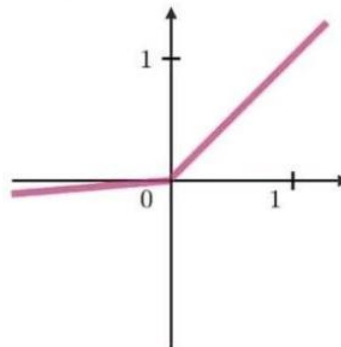
$$g(z) = \frac{1}{1 + e^{-z}}$$



Leaky ReLU

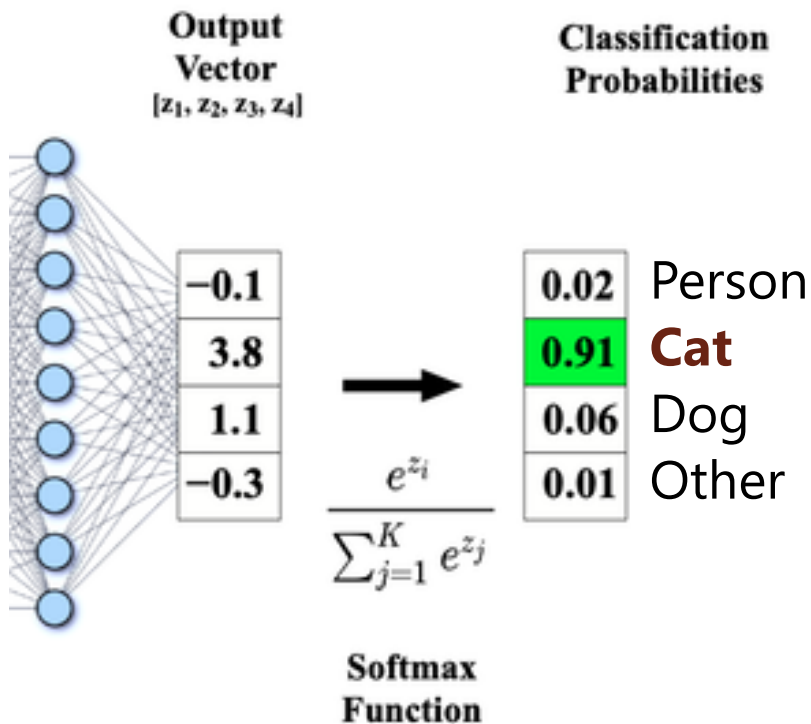
$$g(z) = \max(\epsilon z, z)$$

with $\epsilon \ll 1$



[12.nn\02_activation_functions.ipynb](#)

Softmax activation function



Given a vector of scores $z=(z_1, z_2, \dots, z_n)$, the **softmax** function computes the probabilities $p=(p_1, p_2, \dots, p_n)$ for each class i as:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

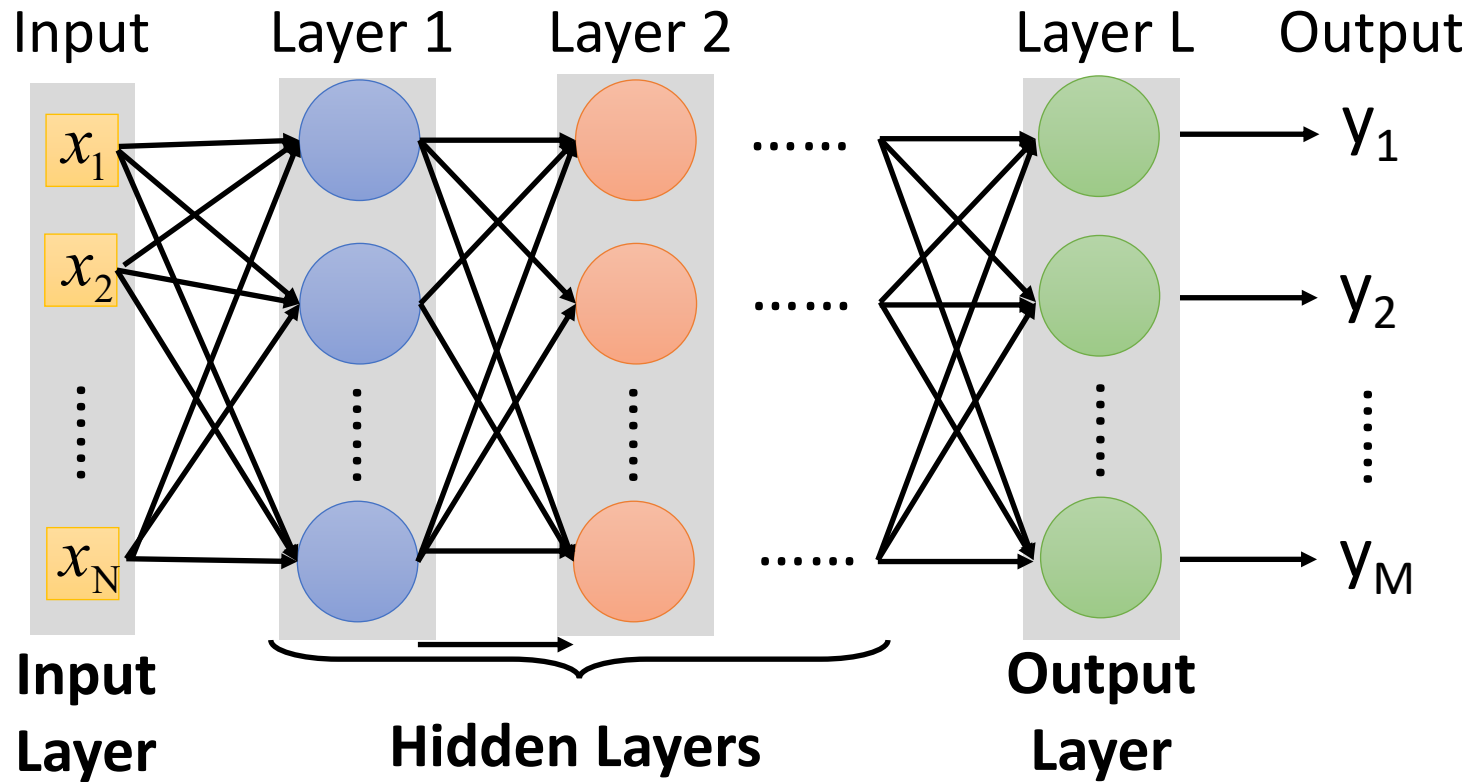
The denominator normalizes across all classes, ensuring probabilities sum to 1

Output Activation & Loss Function Summary

Problem Type	Last Layer Activation	Loss Function
Binary classification	Sigmoid	binary_crossentropy
Multiclass classification	Softmax	categorical_crossentropy
Regression to arbitrary value	[None]	mse
Regression to values between 0 and 1	Sigmoid	mse or binary_crossentropy

Building a NN with Neurons

NN Training: Forward + Backward Pass



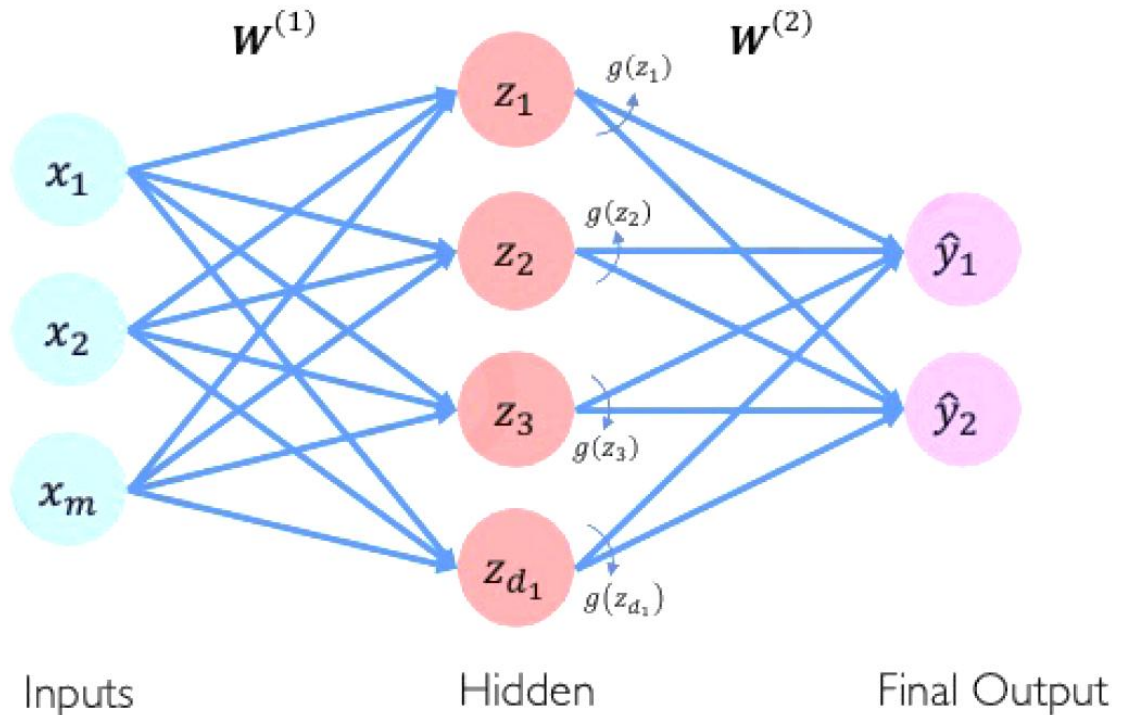
Training repeats two steps for each batch:

1. **Forward pass** → Make a Prediction. Input data flows layer by layer from input → hidden layers → output
2. **Backward pass** → Learn from the Error. Propagate the error backward from output → hidden layers to update weights and biases using gradient descent

Single Layer Neural Network

Key points

- Only one layer of weights between input and output
- Each input x_i is multiplied by a weight w_i
- Weighted sum + bias is passed through an activation function

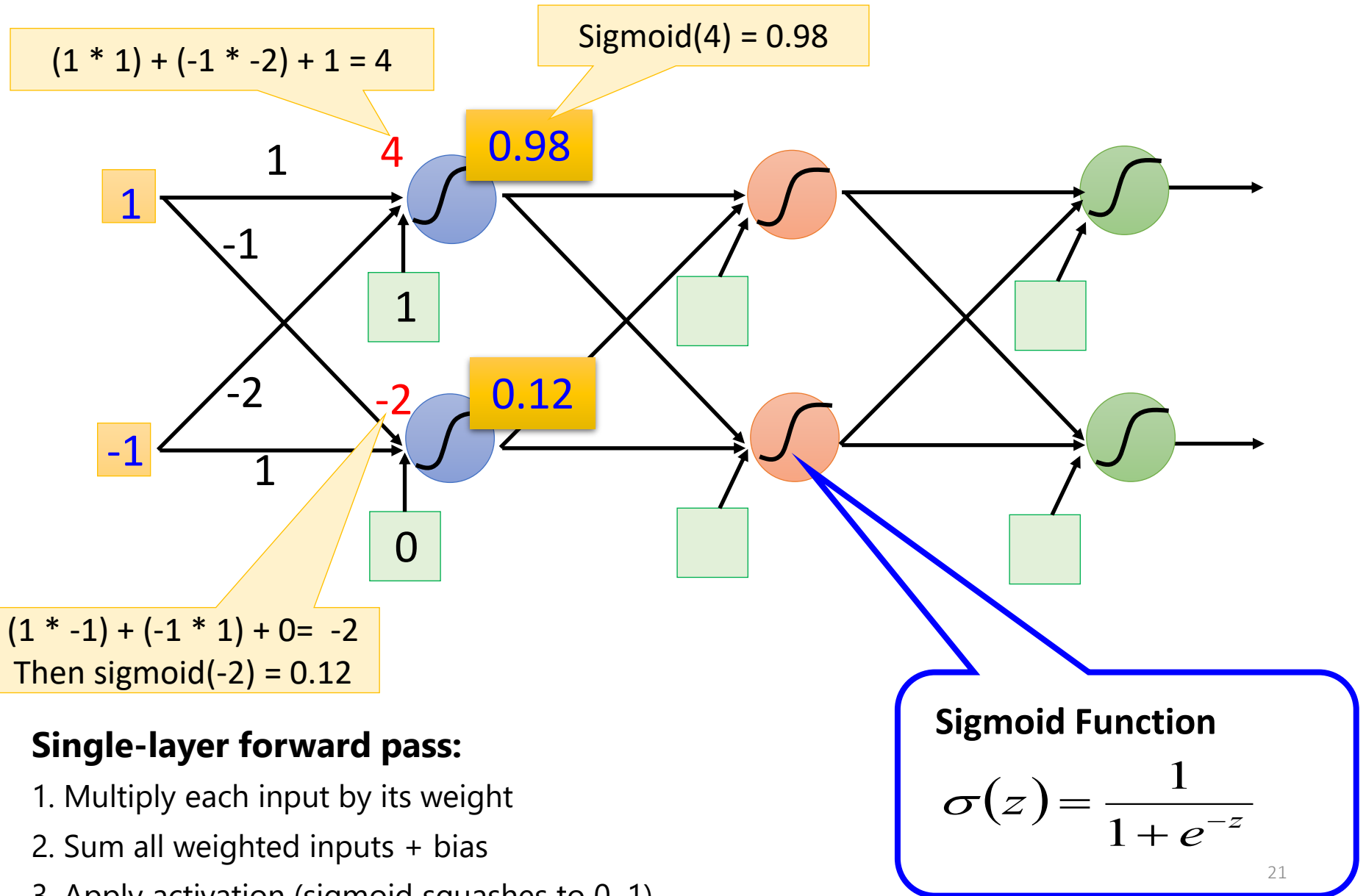


$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^m x_j w_{j,i}^{(1)}$$

$$\hat{y}_i = g \left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,i}^{(2)} \right)$$

w_0 refers to the bias term

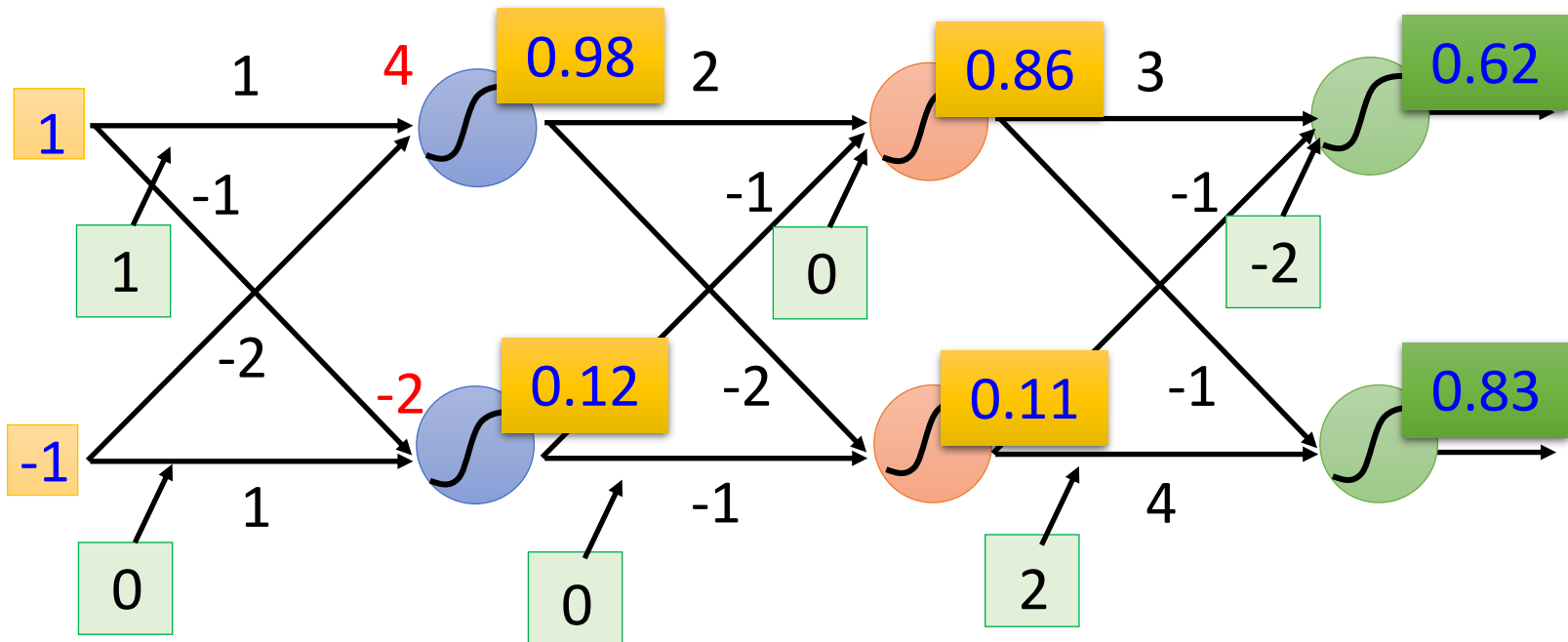
Fully-connected Feedforward Network



Single-layer forward pass:

1. Multiply each input by its weight
2. Sum all weighted inputs + bias
3. Apply activation (sigmoid squashes to 0–1)

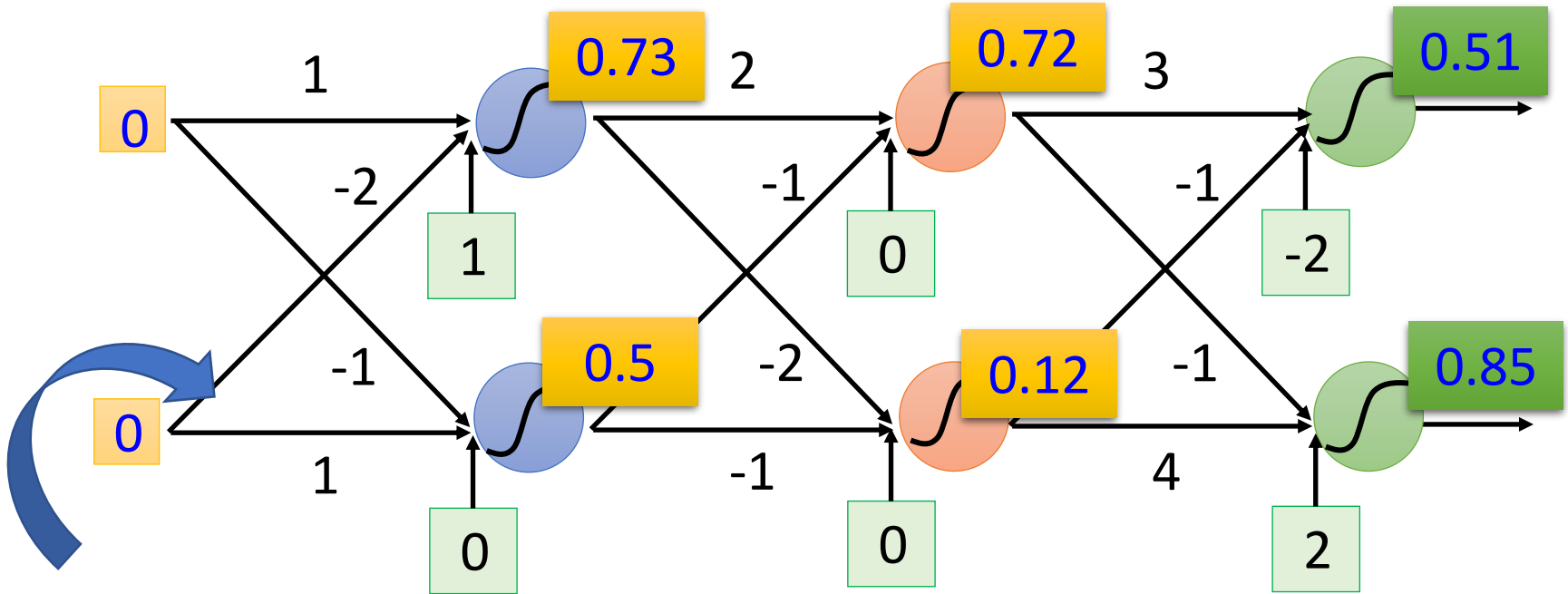
Fully-connected Feedforward Network (with input 1,-1)



Multi-layer forward pass (input [1, -1])

- Each neuron computes: weighted sum of inputs + bias → sigmoid activation.
- Layer 1 outputs become Layer 2 inputs, and so on — this is the **forward pass**.
- Different inputs produce different outputs, but the weights stay fixed until training updates them.

Fully-connected Feedforward Network (with input 0, 0)



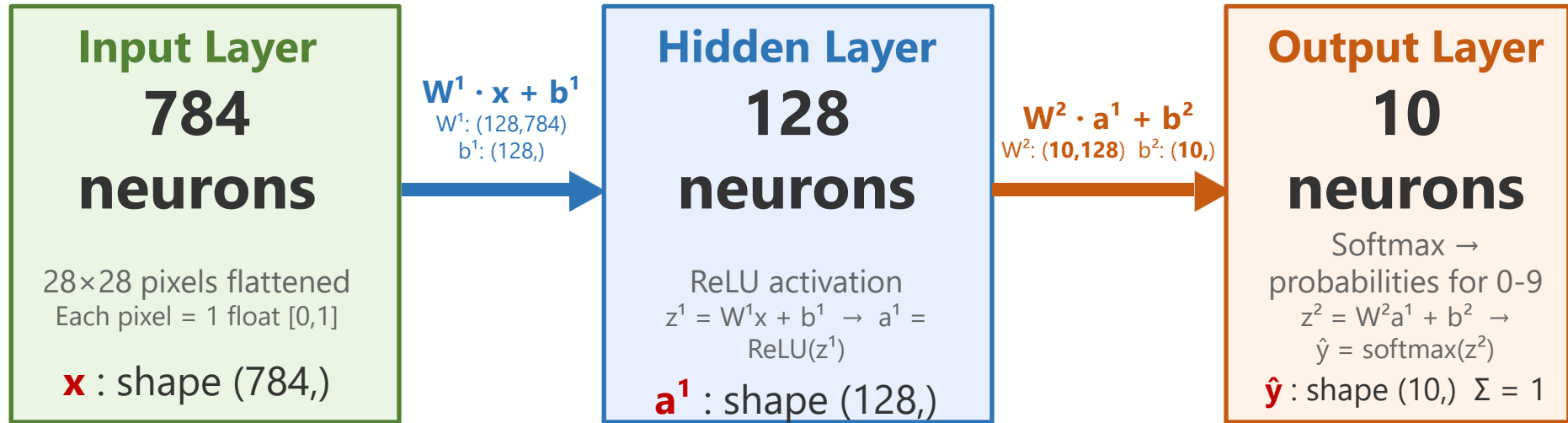
- A neural network is a function
- It maps an input vector to an output vector

$$f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix}$$

$$f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$$

2-Layer Network for MNIST Digit Recognition

Classifying 28x28 handwritten digit images into 10 classes (0-9)



Tensor Summary

- $\mathbf{W}^1$: (128, 784) = 100,352 weights $\mathbf{b}^1$: (128,) = 128 biases
- $\mathbf{W}^2$: (10, 128) = 1,280 weights $\mathbf{b}^2$: (10,) = 10 biases
- Total trainable parameters: **101,770** (99% in $\mathbf{W}^1$ — input-to-hidden dominates)
- Batch input shape: (batch_size, 784) → hidden: (batch_size, 128) → output: (batch_size, 10)

Vectorized Forward Pass

Every operation is a matrix multiply + vector add — no loops over individual neurons

Layer 1 (Hidden)

Pre-activation:

$$\mathbf{z}^1 = \mathbf{W}^1 \cdot \mathbf{x} + \mathbf{b}^1$$

Dimensions:

$$\mathbf{W}^1 \in \mathbb{R}^{128 \times 784}$$

$$\mathbf{x} \in \mathbb{R}^{784}$$

$$\mathbf{b}^1 \in \mathbb{R}^{128}$$

$$\mathbf{a}^1 \in \mathbb{R}^{128} \text{ (128 features)}$$

Activation:

$$\mathbf{a}^1 = \text{ReLU}(\mathbf{z}^1)$$

Layer 2 (Output)

Pre-activation:

$$\mathbf{z}^2 = \mathbf{W}^2 \cdot \mathbf{a}^1 + \mathbf{b}^2$$

Dimensions:

$$\mathbf{W}^2 \in \mathbb{R}^{10 \times 128}$$

$$\mathbf{a}^1 \in \mathbb{R}^{128}$$

$$\mathbf{b}^2 \in \mathbb{R}^{10}$$

$$\hat{\mathbf{y}} \in \mathbb{R}^{10} \text{ (10 probabilities)}$$

Output:

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^2)$$

Why vectorize? Matrix operations run in parallel on GPUs. Instead of looping over 784 inputs one-by-one, a single matrix multiply computes all 128 hidden activations at once. For a batch of N images, replace $\mathbf{x}$ with $\mathbf{X} \in \mathbb{R}^{N \times 784}$ and get all outputs in one operation.

Dimension matching rule: In $\mathbf{W} \cdot \mathbf{x}$, the number of **columns of $\mathbf{W}$** must equal the number of **rows of $\mathbf{x}$** . This is why $\mathbf{W}^1$ is 128×784 (784 inputs $\rightarrow$ 128 outputs) and $\mathbf{W}^2$ is 10×128 (128 $\rightarrow$ 10).

Rules & Guidelines for Network Design

Practical rules for building a 2-layer fully-connected network

1. Input Layer Size

Must match the number of features.

For MNIST: $28 \times 28 = 784$ pixels, so the input layer has 784 neurons. Each pixel value is normalized to $[0, 1]$.

2. Output Layer Size

Must match the number of classes.

For digit recognition: 10 classes (digits 0–9), so the output layer has 10 neurons, each outputting a probability.

3. Hidden Layer Width

A hyperparameter you choose.

Common choices: 64, 128, 256. Start small, scale up if underfitting.

Wider = more capacity to learn complex patterns, but risks overfitting.

4. Activation Functions

Hidden layers: ReLU (fast, avoids vanishing gradients).

Output layer: Softmax (converts raw scores into probabilities that sum to 1 across all classes).

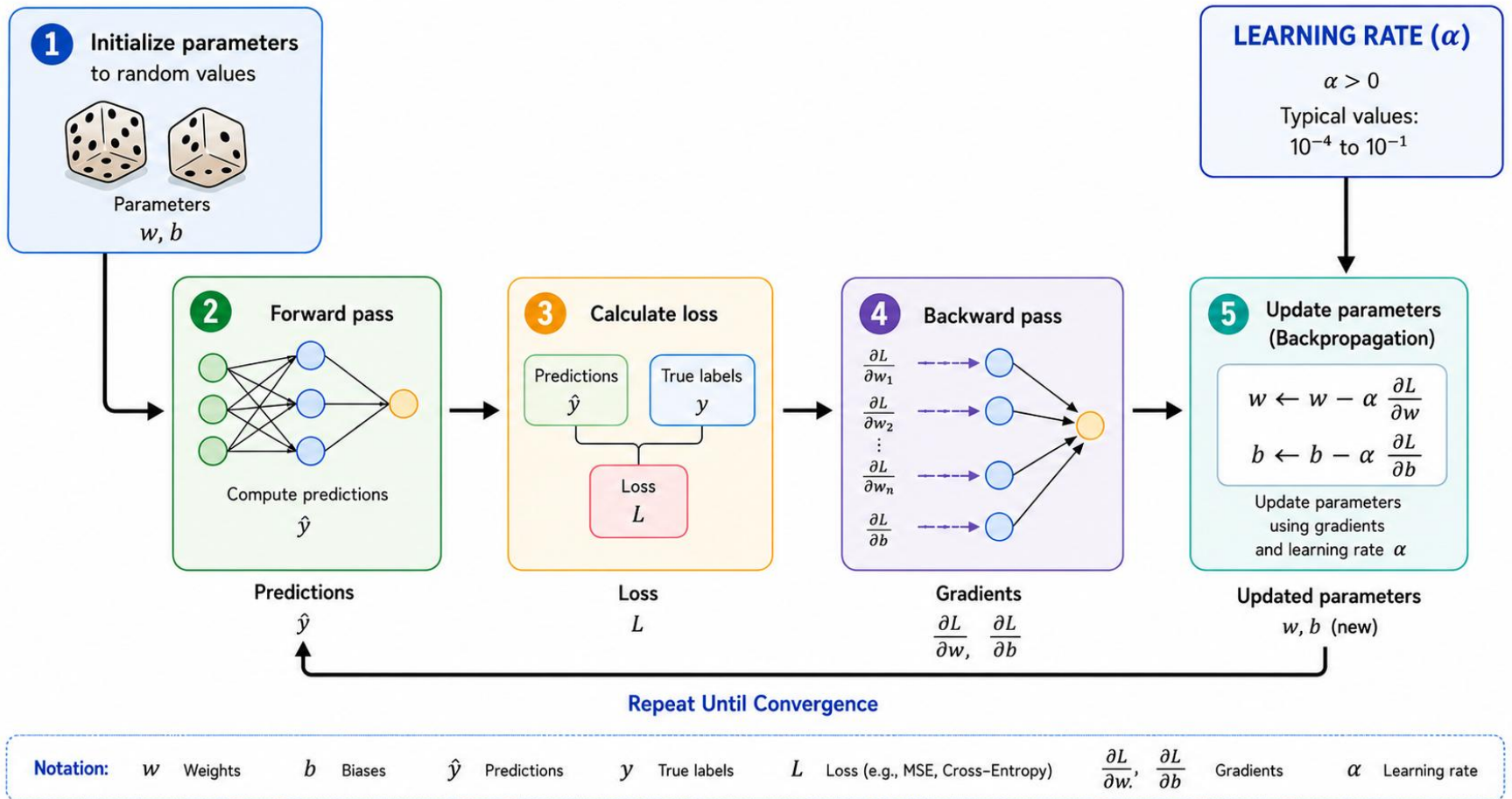
5. Weight Initialization

Random small values. Never initialize all weights to zero.

6. Loss Function

Cross-entropy loss for multi-class classification. It measures how far the predicted label from the true label.

The architecture is fixed before training. Learning only adjusts the weights and biases.



Training Loop

The Full Training Loop

Putting it all together: how a network learns from data over multiple iterations

1 Forward Pass

For each batch of training examples:

- Feed inputs through all layers
- Store intermediate values (z , a) at each layer
- Produce predictions $\hat{y}$ at the output

repeat ↑

2 Compute Loss

Measure prediction error:

- Cross-entropy for classification
- MSE for regression
- Average loss over the batch

3 Backpropagation

Compute gradients via chain rule:

- Start from loss, propagate backward
- Compute $\partial L / \partial W$ and $\partial L / \partial b$ for every layer
- Use stored z , a values from forward pass

4 Update Weights

Gradient descent step (implemented by the **Optimizer**):

$$W \leftarrow W - \alpha \cdot \partial L / \partial W \text{ for all layers}$$

$$b \leftarrow b - \alpha \cdot \partial L / \partial b \text{ for all layers}$$

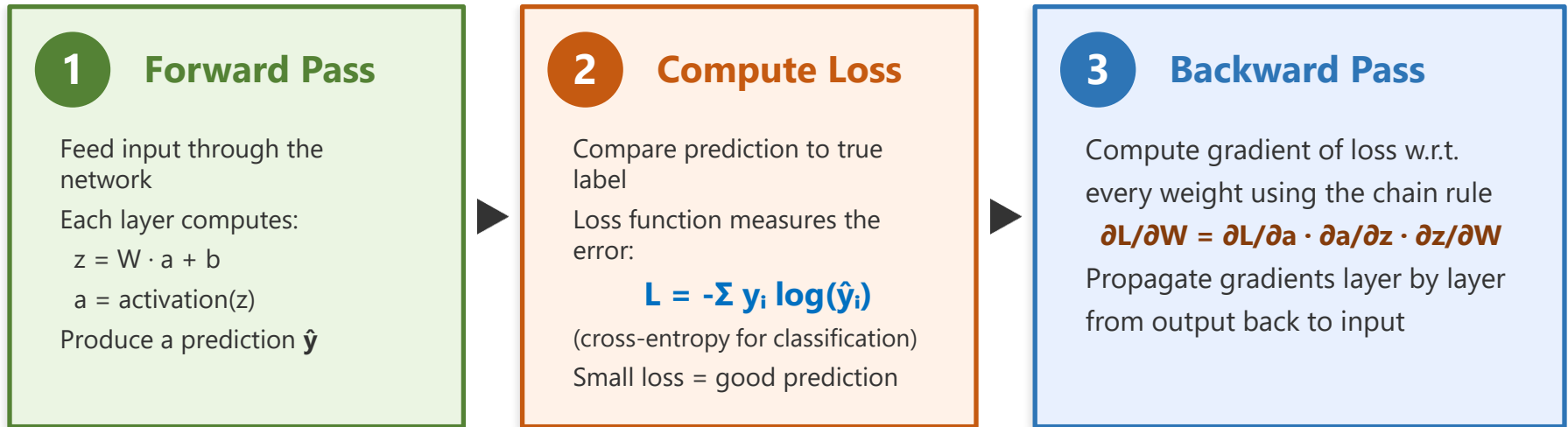
- Learning rate α controls step size

Training terminology

- **Batch**: a subset of training examples processed together (e.g., 32 or 64 images)
- **Iteration**: one forward + backward + update cycle on a single batch
- **Epoch**: one complete pass through all training data (many iterations)
- Training typically runs for 10–100+ epochs until loss converges

What Is Backpropagation?

The algorithm that teaches a neural network by propagating errors backward through every layer



4 Weight Update (Gradient Descent): $W \leftarrow W - \alpha \cdot \partial L / \partial W$ where α is the learning rate

Repeat this cycle for every batch of training data. Each iteration nudges all weights toward lower loss.

Why it works

- The chain rule decomposes a complex gradient into simple local derivatives at each layer
- Each layer only needs its own local gradient and the error signal from the layer above
- This makes backpropagation efficient: $O(n)$ in number of parameters, same cost as a forward pass
- Without it, we'd need to perturb each weight individually — impossibly slow for millions of parameters

The Chain Rule in Backpropagation

How gradients flow backward through the computation graph

The Chain Rule

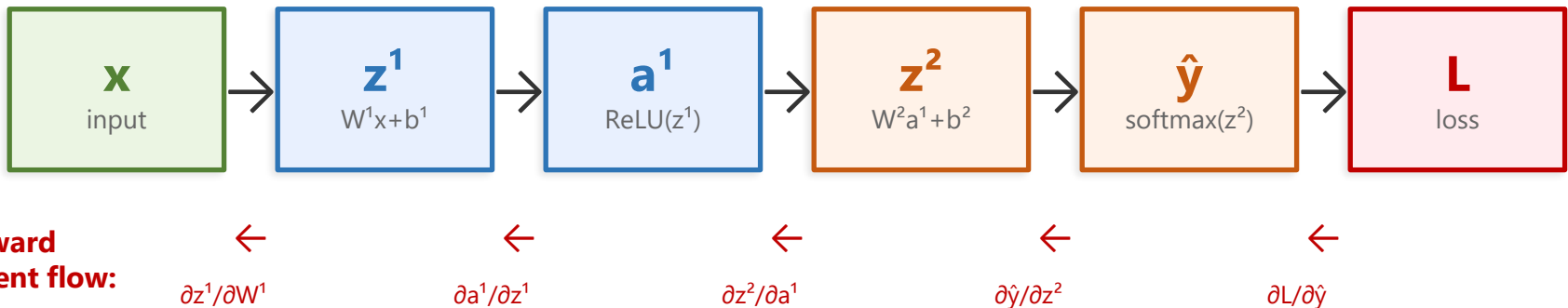
If **L** depends on **a**, which depends on **z**, which depends on **W**, then:

L = loss | **a** = activation (ReLU output) | **z** = pre-activation ($Wx + b$) | **W** = weights

$$\partial L / \partial W = \partial L / \partial a \times \partial a / \partial z \times \partial z / \partial W$$

Each factor is a **local derivative** — computed at one node, then multiplied together along the path from L to W.

2-Layer Example: Gradient of Loss w.r.t. W^1 (first-layer weights)



Full gradient for W^1 : $\partial L / \partial W^1 = \partial L / \partial \hat{y} \times \partial \hat{y} / \partial z^2 \times \partial z^2 / \partial a^1 \times \partial a^1 / \partial z^1 \times \partial z^1 / \partial W^1$

Each factor is computed locally at its node. Multiplying them gives the total gradient — no need to differentiate the entire network at once.

Backpropagation: Worked Example

A minimal 1-neuron example: input $x=2$, weight $w=0.5$, bias $b=1$, target $y=1$, sigmoid activation

Step 1: Forward Pass

Pre-activation:

$$z = w \cdot x + b = 0.5 \times 2 + 1 = \mathbf{2.0}$$

Activation (sigmoid):

$$a = \sigma(2.0) = 1/(1+e^{-2}) = \mathbf{0.88}$$

Loss (MSE):

$$L = \frac{1}{2}(y - a)^2 = \frac{1}{2}(1 - 0.88)^2 = \mathbf{0.0072}$$

Step 2: Backward Pass

Gradient of loss w.r.t. output:

$$\partial L / \partial a = -(y - a) = -(1 - 0.88) = \mathbf{-0.12}$$

Gradient through sigmoid:

$$\partial a / \partial z = a(1-a) = 0.88 \times 0.12 = \mathbf{0.1056}$$

Gradient w.r.t. weight:

$$\partial z / \partial w = x = \mathbf{2}$$

Step 3: Apply the Chain Rule

$$\partial L / \partial w = \partial L / \partial a \times \partial a / \partial z \times \partial z / \partial w = (-0.12) \times (0.1056) \times (2) = \mathbf{-0.0253}$$

Negative gradient $\rightarrow$ increasing w will decrease the loss (move prediction closer to target)

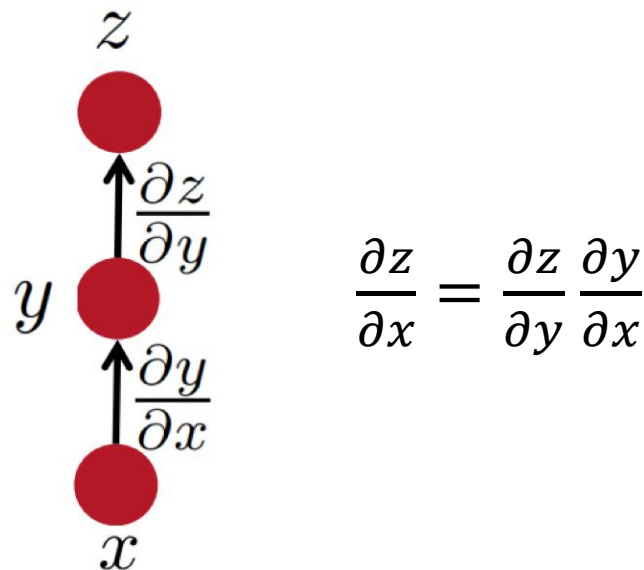
Step 4: Update the Weight ($\alpha = 0.1$)

$$w_{\text{new}} = w - \alpha \cdot \partial L / \partial w = 0.5 - 0.1 \times (-0.0253) = \mathbf{0.5025}$$

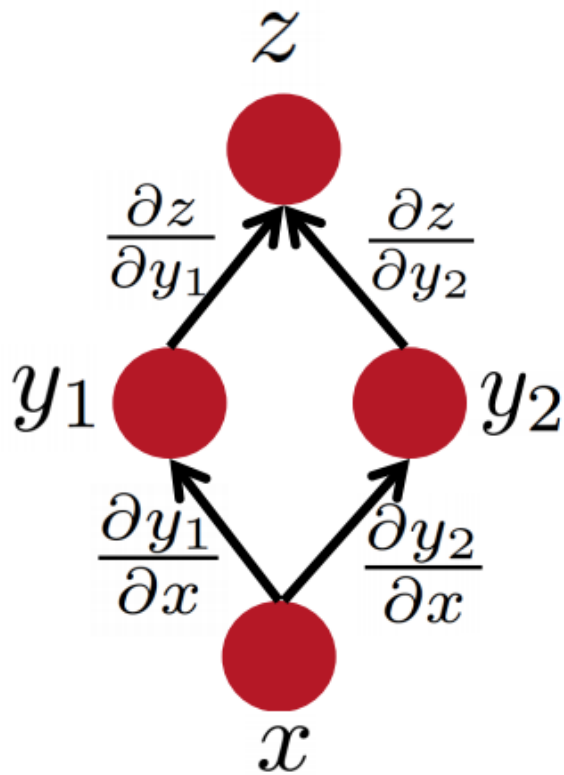
The weight increased slightly ($0.5 \rightarrow 0.5025$), which will push the prediction closer to 1.0 on the next forward pass.

Simple chain rule

- If z is a function of y , and y is a function of x
 - Then z is a function of x , as well
- Question: how to find $\frac{\partial z}{\partial x}$

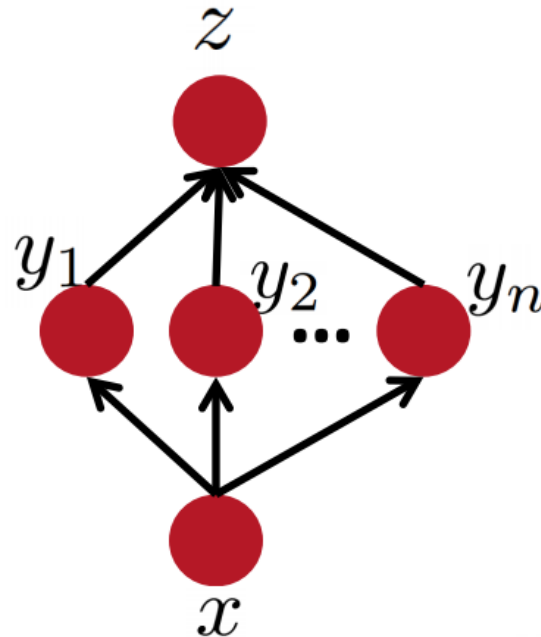


Multiple path chain rule



$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y_1} \frac{\partial y_1}{\partial x} + \frac{\partial z}{\partial y_2} \frac{\partial y_2}{\partial x}$$

In general:



$$\frac{\partial z}{\partial x} = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}$$

Which Loss Function to use?

Match your loss to your task — the wrong choice leads to poor training

A loss function measures how far the model's predictions are from the actual targets

REGRESSION

MSE — Mean Squared Error

$$\text{Loss} = (1/n) \sum (\hat{y}_i - y_i)^2$$

- Predicting continuous values (price, score)
- Penalizes large errors heavily (squared term)
- Default choice for regression

`torch.nn.MSELoss()`

MAE — Mean Absolute Error

$$\text{Loss} = (1/n) \sum |\hat{y}_i - y_i|$$

- Robust to outliers — treats all errors equally
- Best for noisy data with outliers

`torch.nn.L1Loss()`

CLASSIFICATION

Cross-Entropy Loss

$$\text{Loss} = -\sum y_i \cdot \log(\hat{y}_i)$$

- Multi-class: one label per sample
- Standard for softmax output
- Expects raw logits (no softmax needed)

`torch.nn.CrossEntropyLoss()`

Binary Cross-Entropy (BCE)

$$\text{Loss} = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

- Binary (yes/no) or multi-label tasks
- Expects raw logits (includes sigmoid internally)

`torch.nn.BCEWithLogitsLoss()`

Pro tip: Start with **CrossEntropyLoss** for classification or **MSELoss** for regression. Only switch when you have a specific reason.

Loss Functions in PyTorch

Practical examples showing how to compute and use different losses

Regression

```
import torch
import torch.nn as nn

# Sample data
pred = torch.tensor([2.5, 0.3, 1.8])
target = torch.tensor([3.0, 0.0, 2.0])

# MSE Loss
mse = nn.MSELoss()
loss = mse(pred, target)
# loss = 0.1133

# MAE (L1) Loss
mae = nn.L1Loss()
loss = mae(pred, target)
# loss = 0.2667
```

Classification

```
import torch
import torch.nn as nn

# 3-class classification
logits = torch.tensor(
    [[2.0, 0.5, 0.3]])
labels = torch.tensor([0])

# Cross-Entropy (most common)
ce = nn.CrossEntropyLoss()
loss = ce(logits, labels)
# loss = 0.3326

# Binary classification
logits_b = torch.tensor([1.5])
labels_b = torch.tensor([1.0])

bce = nn.BCEWithLogitsLoss()
loss = bce(logits_b, labels_b)
# loss = 0.2014
```

The Role of the Optimizer

The optimizer determines **how** the network's weights are updated after each backward pass

Core Idea: Gradient Descent

- Compute the **gradient** of the loss with respect to each weight (via backpropagation)
- Move each weight in the **opposite direction** of the gradient to reduce loss
- Update rule: $w = w - lr * \text{gradient}$

Key Hyperparameters

- **Learning rate (lr)** — step size; too large = divergence, too small = slow convergence
- **Momentum** — **accumulates past gradients** to smooth updates and escape local minima

Why the Choice Matters

- Different optimizers converge at different speeds and reach different solutions
- The right optimizer can mean the difference between a model that trains in hours vs. days

Common Optimizers Compared

SGD (Stochastic Gradient Descent)

- Simplest optimizer — updates weights using raw gradients
- Can be noisy and slow to converge
- Add **momentum** to smooth noisy gradients and accelerate convergence
- **Best for:** large-scale vision models (with momentum)

Adam (Adaptive Moment)

- Combines **momentum** + **per-parameter learning rates**
- Adapts step size for each weight based on gradient history
- Works well out of the box with default hyperparameters
- **Best for:** general-purpose deep learning, NLP

Momentum — The Intuition

Accumulate past gradients to smooth updates and escape local minima

Without Momentum

- Each update uses only the current gradient — no memory of past steps
- Noisy mini-batch gradients cause **zigzag paths and slow progress**
- Gets **stuck in shallow local minima** — no accumulated force to push through

With Momentum

Accumulates a running average of past gradients — builds a **velocity**

- Consistent gradient directions get reinforced; noise gets canceled out
- Velocity carries updates through flat spots and past shallow traps

How Momentum Escapes Local Minima

Think of a ball rolling through a hilly landscape:

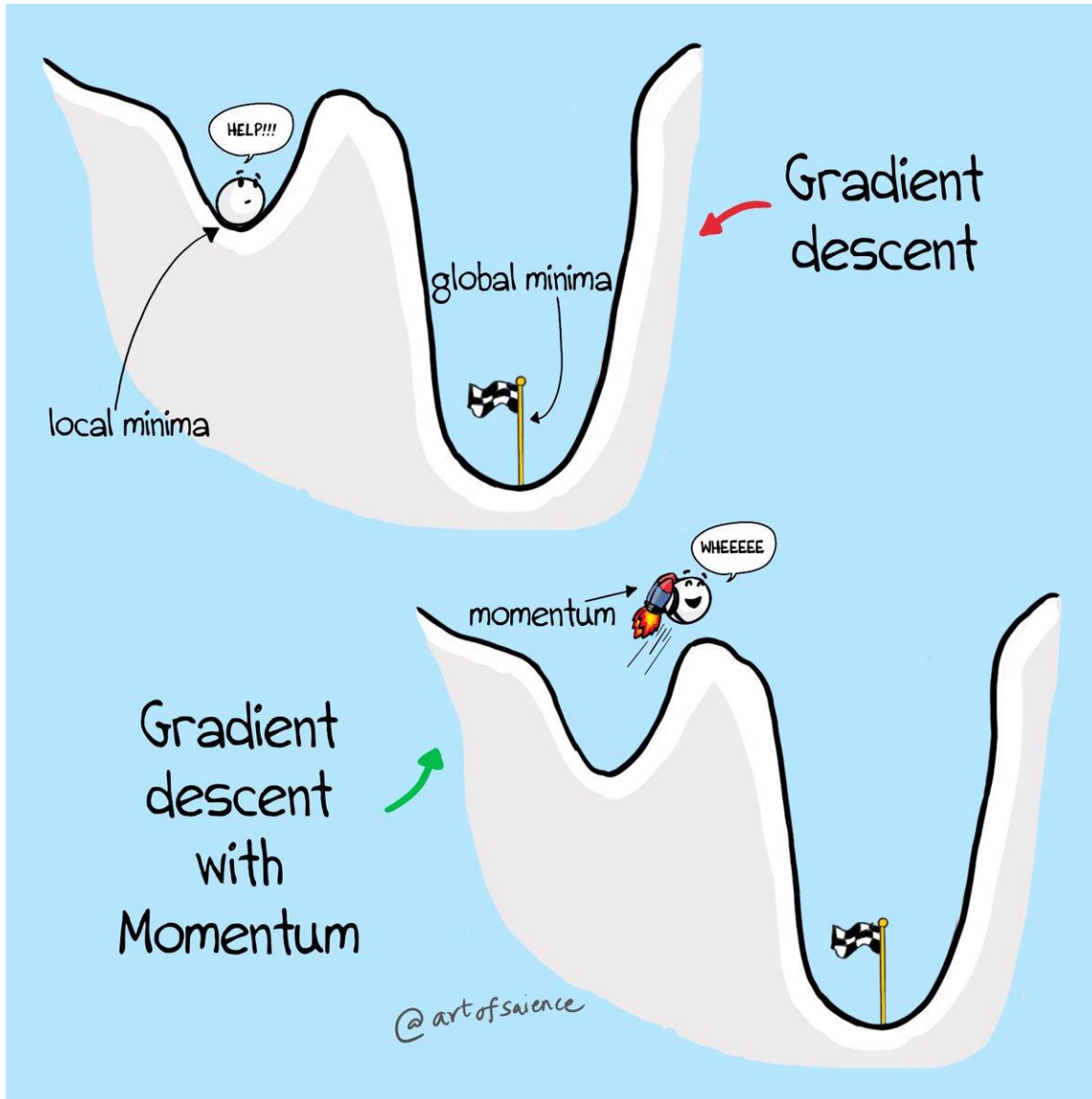
No momentum — the ball stops at the first dip it finds. This might be a shallow, suboptimal minimum. There's no force to push it out.

With momentum — accumulated velocity from past gradients gives the ball enough speed to roll right over shallow dips. It only settles in deeper, better minima where the walls are too steep for its velocity to overcome.

Key takeaway: Momentum = memory. The optimizer builds momentum by using past gradients to *smooth updates, accelerate learning, and escape shallow minima.*

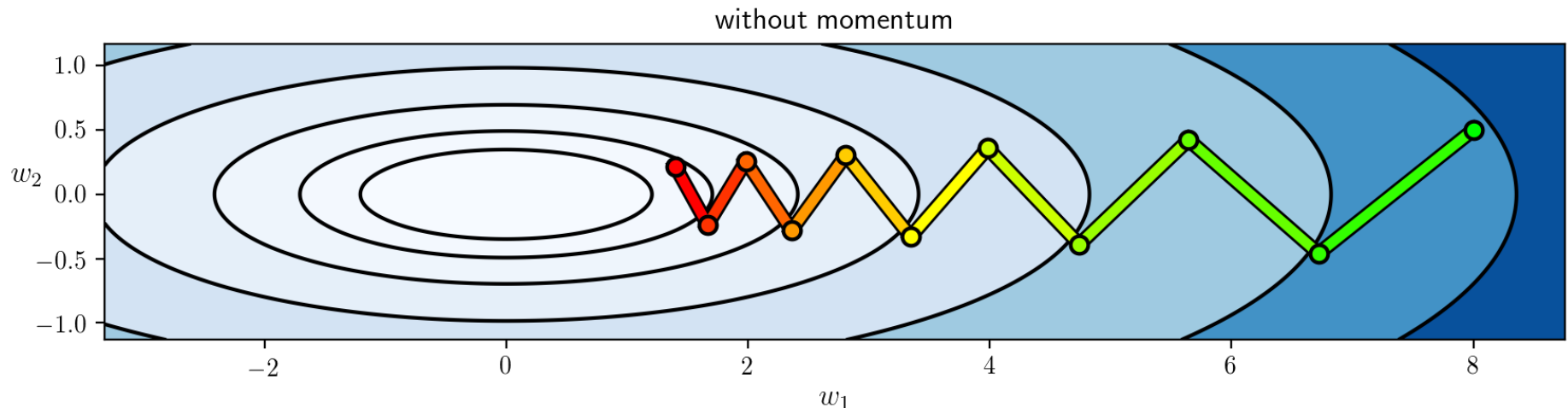
How Momentum Helps Gradient Descent

[Source](#)

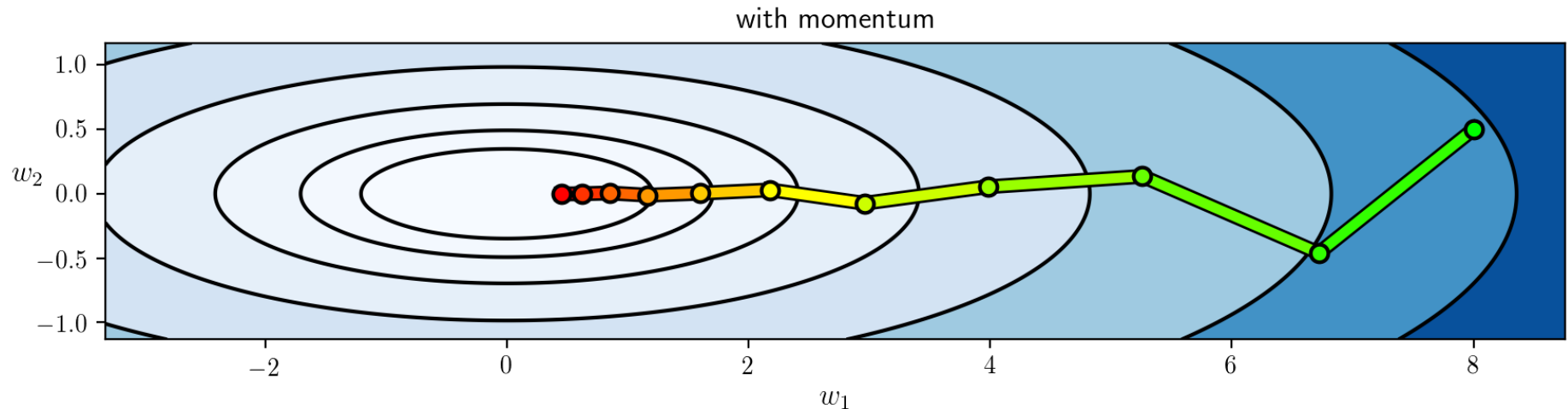


Momentum **accumulates past gradients** into a velocity, **smoothing noisy updates** and accelerating movement in consistent directions—helping the optimizer **escape shallow local minima** that trap vanilla gradient descent

Without vs. With Momentum



- **Without momentum**, gradient descent follows a **noisy, zigzag path** — each step reacts only to the current gradient, making **progress slow and erratic**



- With momentum, **past gradients are accumulated into a velocity term**, **smoothing the trajectory** and **accelerating convergence** toward the minimum

Momentum — How It Works

Two steps per update: blend past velocity with current gradient, then move

The Update Rule

1. $\mathbf{v}_t = \beta \cdot \mathbf{v}_{t-1} + (1 - \beta) \cdot \nabla L$ *blend old velocity with new gradient*

2. $W = W - \alpha \cdot \mathbf{v}_t$ *update weights with smoothed velocity*

β = momentum (0.9) α = learning rate ∇L = current gradient $\mathbf{v}$ = velocity

Example: $\beta = 0.9$, gradients oscillate between +3 and -1

Step	Gradient (∇L)	Velocity ($\mathbf{v}_t$)	What's Happening
0	—	0.00	Start from rest — no velocity yet
1	+3.0	0.30	$0.9 \times 0 + 0.1 \times 3 = 0.30 \rightarrow$ First gradient gives initial push
2	-1.0	0.17	$0.9 \times 0.30 + 0.1 \times (-1) = 0.17 \rightarrow$ Noise slows but doesn't reverse
3	+3.0	0.45	$0.9 \times 0.17 + 0.1 \times 3 = 0.45 \rightarrow$ Reinforced — velocity grows
4	-1.0	0.31	$0.9 \times 0.45 + 0.1 \times (-1) = 0.31 \rightarrow$ Still positive despite noise

Takeaway: Even though gradients flip between +3 and -1, the velocity stays positive (≈ 0.3 – 0.5). Momentum extracts the true downhill signal from noisy oscillations.

Optimizers in PyTorch

Creating Optimizers

```
# Vanilla SGD
optim = torch.optim.SGD(
    model.parameters(), lr=0.01)

# SGD with momentum
optim = torch.optim.SGD(
    model.parameters(), lr=0.01,
    momentum=0.9)

# Adam (most popular default)
optim = torch.optim.Adam(
    model.parameters(), lr=1e-3)
```

Training Loop Pattern

```
for epoch in range(num_epochs):
    for X, y in train_loader:
        pred = model(X)
        loss = criterion(pred, y)

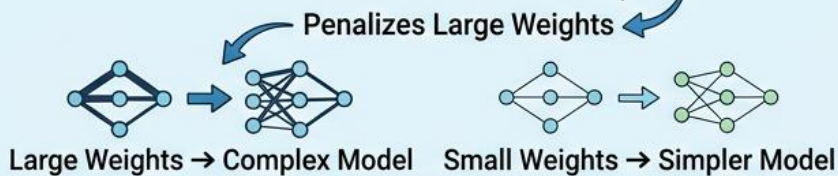
        optim.zero_grad()
        loss.backward()
        optim.step()
```

L2 REGULARIZATION

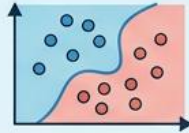


HOW L2 WORKS: WEIGHT DECAY

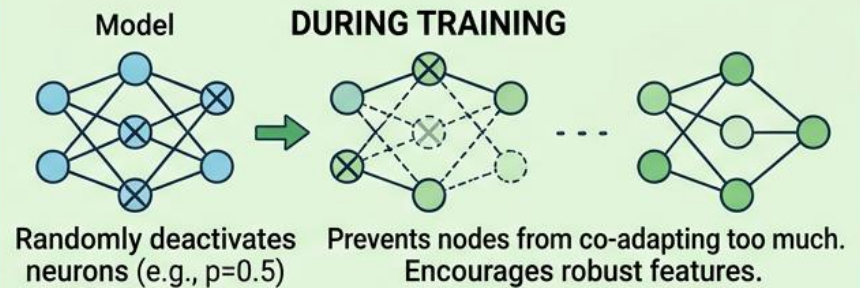
$$\text{Total Loss} = \text{Loss}(\text{Data}, \text{Model}) + \lambda \cdot \sum_i^M (\text{Weights}^2)$$



Keeps weights small, creating smoother decision boundaries and a less complex network.



DROPOUT



- KEY BENEFITS
- ✓ Prevents Overfitting
 - ✓ Improves Generalization
 - ✓ Creates more robust models

Regularization Techniques

The Overfitting Problem

A model overfits when it memorizes training data instead of learning general patterns

1. **Symptoms** — high training accuracy but poor validation/test performance
2. **Root cause** — model capacity exceeds what the data complexity requires
3. **Training loss** keeps decreasing while **validation loss** starts increasing
4. **Result** — the model fails to generalize to unseen data

Regularization techniques constrain the model to prevent overfitting

Regularization Techniques Overview

Regularization adds constraints to prevent the model from becoming too complex

1. **L1 Regularization (Lasso)** — adds sum of absolute weights to loss; promotes sparsity
2. **L2 Regularization (Ridge)** — adds sum of squared weights to loss; penalizes large weights
3. **Dropout** — randomly deactivates neurons during training; forces redundancy
4. **Early Stopping** — halt training when validation loss stops improving
5. **Data Augmentation** — increase effective dataset size with transformations

L2 Regularization in PyTorch

PyTorch implements L2 regularization via the **weight_decay** parameter in optimizers

```
# L2 regularization with weight_decay
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001,
    weight_decay=1e-4 # lambda for L2 penalty
)
```

Key insight — `weight_decay=1e-4` means every weight is pulled toward zero by 0.01% each step

Dropout — How It Works

Randomly silence neurons during training so the network learns robust features

During Training

- Each forward pass, randomly set neurons to zero with probability p
- Every step trains a different sub-network
- Typical $p = 0.2-0.5$ depending on layer size

During Inference

- All neurons are active — no dropout applied
- Outputs scaled by $(1-p)$: multiply the outputs by $(1-p)$ so turning off dropout doesn't suddenly make the network's outputs larger
- Result: The model behaves like an ensemble of all dropout sub-networks combined

Benefit: Stops the network from relying on specific neurons and pushes it to learn features that work in many situations.

With Dropout

B is randomly absent, so A must learn to be useful on its own. Each neuron develops independently meaningful features — the network becomes robust to missing activations.

Why dropout works — three effects:

1. **Ensemble effect** — averages predictions from thousands of different sub-networks
2. **Breaks co-adaptation** — every neuron is forced to learn independently on its own, no free-riders
3. **Implicit regularization** — effect similar to L2, keeps weights small and model simple

Regularization using Pytorch

Model Definition

```
class RegularizedNet(nn.Module):
    def __init__(self, in_d, hid, out_d):
        super().__init__()
        self.fc1 = nn.Linear(in_d, hid)
        self.drop1 = nn.Dropout(0.5)
        self.fc2 = nn.Linear(hid, hid)
        self.drop2 = nn.Dropout(0.3)
        self.fc3 = nn.Linear(hid, out_d)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = self.drop1(x)
        x = F.relu(self.fc2(x))
        x = self.drop2(x)
        return self.fc3(x)
```

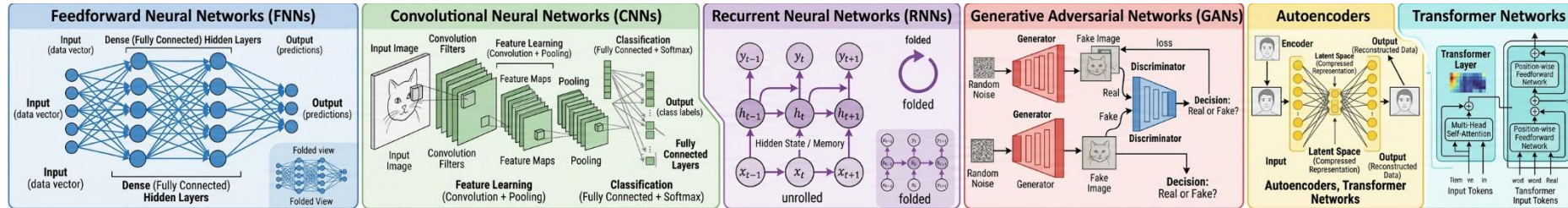
Training Loop

```
model = RegularizedNet(784, 256, 10)
opt = torch.optim.Adam(
    model.parameters(), lr=1e-3,
    weight_decay=1e-4) # L2 regularization
loss_fn = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    model.train() # dropout ON
    for xb, yb in train_loader:
        loss = loss_fn(model(xb), yb)
        opt.zero_grad()
        loss.backward()
        opt.step()

    model.eval() # dropout OFF
    with torch.no_grad():
        val_loss = loss_fn(
            model(X_val), y_val)
```

- **weight_decay** in the optimizer handles L2 regularization automatically
- **model.train()** / **model.eval()** toggles dropout on/off between training and inference



NN Architectures

Neural Network Architectures (1 of 2)

Each architecture connects neurons differently to solve different problems

Feedforward (FNN)

a.k.a. Multilayer Perceptron (MLP)

Data flows one direction — input → hidden layers → output. No loops or cycles.

Real-World Uses

- Credit scoring & loan approvals
- Customer churn prediction
- Spam detection & email filtering
- House price prediction

Convolutional (CNN)

The “eyes” of deep learning

Slides small filters across images/grids to detect local patterns (edges → shapes → objects).

Real-World Uses

- Medical imaging (tumors in MRI/CT)
- Self-driving car vision
- Facial recognition & video analysis
- Satellite imagery analysis

Recurrent (RNN)

Loops output back as input, giving the network memory of previous steps in a sequence.

Real-World Uses

- Speech recognition & voice assistants
- Stock price & weather forecasting
- Music generation & composition
- Sensor data & time-series analysis

Neural Network Architectures (2 of 2)

Generative Adversarial (GAN)

Two networks compete — a **generator** creates fakes, a **discriminator** tries to spot them. Both improve through competition.

Real-World Uses

- Photorealistic image & art generation
- Data augmentation for medical imaging
- Style transfer & image super-resolution

Autoencoder

An **encoder** compresses input to a compact code; a **decoder** reconstructs from it. Learns the most important features automatically.

Real-World Uses

- Fraud & anomaly detection in banking
- Image denoising & compression
- Drug discovery (molecular features)

Transformer — Powers virtually all modern AI

Uses **self-attention** to relate all input elements at once, allowing the model to capture long-range relationships and scale efficiently with more data and compute.

Self-attention answers: Which words (or pixels) matter most for understanding the input?

Real-World Uses

- **Conversational AI** and **Code generation** — ChatGPT, Claude, Gemini
- **Text-to-image** — DALL-E, Midjourney, Stable Diffusion
- **Search & translation** — Google Search, Google Translate

Summary: The Neural Network Pipeline

1 Neurons & Activation

- Inputs $\times$ **weights** + **bias** — the fundamental computation unit
- **Activation functions** (ReLU, Sigmoid, Softmax) introduce non-linearity
- Without activation, networks reduce to linear models

2 Network Architecture

- Layers of neurons form **feedforward networks**
- Deeper networks learn **hierarchical features** from raw data
- CNNs, RNNs, Transformers extend the basic architecture

3 Loss Functions

- Quantify how far predictions are from truth
- **Regression:** MSE, MAE
- **Classification:** Cross-Entropy, BCE

6 Regularization

- **L2 penalty** constrains weight magnitudes
- **Dropout** randomly disables neurons during training
- **Early Stopping** halts training before overfitting

5 Optimizers

- Update weights to **minimize loss**
- **Adam / AdamW:** fast convergence, best defaults
- **SGD + momentum:** better generalization

4 Backpropagation

- **Chain rule** computes gradients layer by layer
- Error signal flows **backward** to update every weight
- Enables efficient gradient computation for deep networks

Build neurons → Stack into networks → Define loss → Backpropagate → Optimize → Regularize to generalize

Resources

- [Practical Deep Learning for Coders](#)
- [Machine Learning with PyTorch and Scikit-Learn](#)
- [Dive into Deep Learning](#)
- [LLM Chronicles](#)
- [Towards Data Science](#)